Relief Valve Sizing with Units and Engineering Handoff

This example derives ideal-gas relief-valve sizing equations, checks units before numeric handoff, generates reusable MATLAB® functions, and reuses those functions in a compact upset-scenario screening workflow.

Workflow: compressible-flow assumptions -> symbolic mass flux -> choked-flow criterion -> required area -> unit audit -> generated functions -> standard orifice selection -> scenario screening and margin review.

Requires: MATLAB R2025a or later, Symbolic Math Toolbox™. Optional: Simscape™ Fluids™.

Copyright 2026 The MathWorks, Inc.

1. Define a representative gas relief case

Use a representative isobutane vapor case for a pressurized process vessel. The scenario is chosen to exercise the unit decisions that matter in relief calculations: gauge versus absolute pressure, absolute temperature, molecular weight, mass-flow units, and area units.

```
u = symunit;
relievingPressureWithUnits = 150*u.psi + 1*u.atm;
backPressureWithUnits = 15*u.psi + 1*u.atm;
temperatureWithUnits = 120*u.Fahrenheit;
requiredMassFlowWithUnits = 10000*u.lbm/u.hr;
molecularWeightWithUnits = 58.12*u.g/u.mol;
heatCapacityRatioVal = 1.09;
dischargeCoefficientVal = 0.975;
sizingMarginVal = 0.10;
gasConstantWithUnits = 8.31446261815324*u.J/(u.mol*u.K);
```

Gauge pressure is represented as a pressure difference plus atmospheric pressure. Temperature is converted as an absolute temperature, not as a temperature difference.

```
relievingPressureSI = unitConvert(relievingPressureWithUnits, u.Pa);
backPressureSI = unitConvert(backPressureWithUnits, u.Pa);
temperatureSI = unitConvert(temperatureWithUnits, u.K, 'Temperature', 'absolute');
requiredMassFlowSI = unitConvert(requiredMassFlowWithUnits, u.kg/u.s);
molecularWeightSI = unitConvert(molecularWeightWithUnits, u.kg/u.mol);
referenceInputs = table(["Relieving pressure"; "Backpressure"; "Relieving temperature"; ...
```

```
    "Required mass flow"; "Molecular weight"; "Heat-capacity ratio"; "Discharge coefficient"], ...
    [double(separateUnits(relievingPressureSI)); double(separateUnits(backPressureSI)); ...
    double(separateUnits(temperatureSI)); double(separateUnits(requiredMassFlowSI)); ...
    double(separateUnits(molecularWeightSI)); heatCapacityRatioVal; dischargeCoefficientVal], ...
    ["Pa"; "Pa"; "K"; "kg/s"; "kg/mol"; "1"; "1"], ...
    'VariableNames', {'Input', 'SIValue', 'SIUnit'})
```

`referenceInputs = 7×3 table`

	Input	SIValue	SIUnit
1	"Relieving pressure"	1.1355e+06	"Pa"
2	"Backpressure"	2.0475e+05	"Pa"
3	"Relieving temperature"	322.0389	"K"
4	"Required mass flow"	1.2600	"kg/s"
5	"Molecular weight"	0.0581	"kg/mol"
6	"Heat-capacity ratio"	1.0900	"1"
7	"Discharge coefficient"	0.9750	"1"

2. Derive the ideal-gas mass-flux equations

For ideal-gas discharge through an orifice, the subcritical mass flux is written in terms of the downstream-to-upstream pressure ratio $r = P_2/P_1$. The gas constant per unit mass is $R_{specific} = R_u/MW$.

```
syms P1 P2 T1 MW R_u k Cd A mdot_req positive
assume(k > 1)
R_specific = R_u/MW;
pressureRatio = P2/P1;
G_subcritical = P1*sqrt((2*k/(R_specific*T1*(k - 1)))*(pressureRatio^(2/k) - pressureRatio^((k + 1)/k)))
```

`G_subcritical =`

$$\frac{\sqrt{2}\ \sqrt{\mathrm{MW}}\ P_1\ \sqrt{k}\ \sqrt{\left(\frac{P_2}{P_1}\right)^{2/k} - \left(\frac{P_2}{P_1}\right)^{\frac{k+1}{k}}}}{\sqrt{R_u}\ \sqrt{T_1}\ \sqrt{k-1}}$$

```
G_choked = P1*sqrt(k/(R_specific*T1)*(2/(k + 1))^((k + 1)/(k - 1)))
```

G_choked =

$$\frac{\sqrt{\mathrm{MW}}\ P_1\ \sqrt{k}\ \sqrt{\left(\frac{2}{k+1}\right)^{\frac{k+1}{k-1}}}}{\sqrt{R_u}\ \sqrt{T_1}}$$

Keeping both branches symbolic makes the operating-regime assumption visible instead of burying it inside an empirical sizing constant.

3. Derive the choked-flow criterion

Choked flow occurs when the subcritical mass-flux factor reaches its maximum. The pressure-ratio-dependent factor is $\phi(r) = r^{2/k} - r^{(k+1)/k}$.

```
syms r_ratio z positive
phi = r_ratio^(2/k) - r_ratio^((k + 1)/k);
dPhi_dr = diff(phi, r_ratio)
```

dPhi_dr =

$$\frac{2\,r_{\mathrm{ratio}}^{\frac{2}{k}-1}}{k} - \frac{r_{\mathrm{ratio}}^{\frac{k+1}{k}-1}\,(k+1)}{k}$$

```
stationaryCondition = r_ratio^((k - 1)/k) == 2/(k + 1)
```

stationaryCondition =

$$r_{\mathrm{ratio}}^{\frac{k-1}{k}} = \frac{2}{k+1}$$

```
zCrit = solve(1 - (k + 1)/2*z == 0, z);
criticalPressureRatioExpr = simplify(zCrit^(k/(k - 1)))
```

criticalPressureRatioExpr =

$$\left(\frac{2}{k+1}\right)^{\frac{k}{k-1}}$$

```
criticalPressureRatioVal = double(subs(criticalPressureRatioExpr, k, heatCapacityRatioVal))
```

```
criticalPressureRatioVal =
0.5868
```

The result is the familiar critical ratio $r_{crit} = (2/(k+1))^{k/(k-1)}$, so the flow is choked when $P_2/P_1 \le r_{crit}$.

4. Solve symbolically for required area

Mass flow is $\dot{m} = C_d A G$. Because this equation is linear in area, the required area follows directly once the mass-flux branch is selected.

Choked-flow required area:

```
A_choked = simplify(mdot_req/(Cd*G_choked));
```

Subcritical-flow required area:

```
A_subcritical = simplify(mdot_req/(Cd*G_subcritical));
```

Branch-selected expression used for generated sizing functions:

```
G_selected = piecewise(P2/P1 <= criticalPressureRatioExpr, G_choked, G_subcritical);
A_required = simplify(mdot_req/(Cd*G_selected))
```

```
A_required =
```

$$\begin{cases} \dfrac{\sqrt{R_u}\ \sqrt{T_1}\ \mathrm{mdot_{req}}\ (k+1)^{\frac{k+1}{2\,(k-1)}}}{2^{\frac{k+1}{2\,(k-1)}}\,\mathrm{Cd}\ \sqrt{\mathrm{MW}}\ P_1\ \sqrt{k}} & \text{if } P_2 \le P_1 \left(\dfrac{2}{k+1}\right)^{\frac{k}{k-1}} \\ \dfrac{\sqrt{2}\ {P_1}^{\frac{1}{k}-\frac{1}{2}}\ \sqrt{R_u}\ \sqrt{T_1}\ \mathrm{mdot_{req}}\ \sqrt{k-1}}{2\,\mathrm{Cd}\ \sqrt{\mathrm{MW}}\ {P_2}^{\frac{1}{2\,k}}\ \sqrt{k}\ \sqrt{P_1\,{P_2}^{1/k} - {P_1}^{1/k}\,P_2}} & \text{otherwise} \end{cases}$$

5. Audit units before extracting numeric magnitudes

Substitute unit-bearing values into the symbolic area expression first. Only after `checkUnits` and `unitConvert` succeed should the workflow extract numeric magnitudes for generated functions or downstream screening.

```
AReferenceWithUnits = subs(A_choked, [P1 T1 MW R_u k Cd mdot_req], ...
    [relievingPressureSI temperatureSI molecularWeightSI gasConstantWithUnits heatCapacityRatioVal
dischargeCoefficientVal requiredMassFlowSI]);
AReference_m2 = unitConvert(AReferenceWithUnits, u.m^2)
```

```
AReference_m2 =
```

$$\frac{24315451026603666214504017402408053557\ 2^{7/18}\ \sqrt{5}\ \sqrt{7}\ 10^{5/18}\ \sqrt{109}\ 209^{11/18}\ \sqrt{1453}\ \sqrt{26003479}}{153771912985203406160592371712000000000000000000000000}$$

```
areaUnitCheck = checkUnits(AReference_m2 == 1*u.m^2)
```

```
areaUnitCheck =
struct with fields:
    Consistent: 1
    Compatible: 1
```

```
AReferenceVal_m2 = double(separateUnits(AReference_m2))
```

```
AReferenceVal_m2 =
3.9004e-04
```

```
AReferenceVal_in2 = double(separateUnits(unitConvert(AReference_m2, u.in^2)))
```

```
AReferenceVal_in2 =
0.6046
```

This reference case requires about 0.60 in^2 before applying the discrete standard-orifice margin.

6. Quantify common unit mistakes

This section reruns the reference calculation with one unit mistake at a time. The correct calculation uses 150 psig as `150*u.psi + 1*u.atm` and converts 120 degF as an absolute temperature. The comparison rows show how much the required area changes if either setup step is missed.

```
pressureGaugeOnlySI = unitConvert(150*u.psi, u.Pa);
temperatureAsDifference = unitConvert(120*u.Fahrenheit, u.K);
AUsingGaugePressure = unitConvert(subs(A_choked, [P1 T1 MW R_u k Cd mdot_req], ...
    [pressureGaugeOnlySI temperatureSI molecularWeightSI gasConstantWithUnits heatCapacityRatioVal
dischargeCoefficientVal requiredMassFlowSI]), u.in^2);
```

```
AUsingTemperatureDifference = unitConvert(subs(A_choked, [P1 T1 MW R_u k Cd mdot_req], ...
    [relievingPressureSI temperatureAsDifference molecularWeightSI gasConstantWithUnits heatCapacityRatioVal
dischargeCoefficientVal requiredMassFlowSI]), u.in^2);
trapArea_in2 = [AReferenceVal_in2; double(separateUnits(AUsingGaugePressure));
double(separateUnits(AUsingTemperatureDifference))];
table(["Correct absolute pressure and absolute temperature"; ...
    "Mistake: gauge pressure used as absolute"; ...
    "Mistake: Fahrenheit treated as temperature difference"], ...
    trapArea_in2, 100*(trapArea_in2/AReferenceVal_in2 - 1), ...
    'VariableNames', {'Case', 'Area_in2', 'AreaError_pct'})
```

ans = 3×3 table

	Case	Area_in2	AreaError_pct
1	"Correct absolute pressure and absolute temperature"	0.6046	0
2	"Mistake: gauge pressure used as absolute"	0.6638	9.7973
3	"Mistake: Fahrenheit treated as temperature difference"	0.2751	-54.5012

The pressure mistake removes atmospheric pressure from the upstream absolute pressure. The temperature mistake is larger here because `unitConvert` treats Fahrenheit as a temperature interval unless the `Temperature` option is set to `absolute`.

7. Compute symbolic area sensitivities

Start with raw symbolic derivatives of required area with respect to each input. Then normalize each derivative by x/A to form a dimensionless local elasticity: percent required-area change per percent input change around the scenario being evaluated.

```
elasticityVars = [mdot_req P1 P2 T1 MW k Cd];
elasticityLabels = ["Mass flow"; "Upstream absolute pressure"; "Backpressure"; ...
    "Absolute temperature"; "Molecular weight"; "Heat-capacity ratio"; "Discharge coefficient"];
areaGradientChoked = jacobian(A_choked, elasticityVars);
areaGradientSubcritical = jacobian(A_subcritical, elasticityVars);
elasticitiesChoked = simplify(elasticityVars./A_choked.*areaGradientChoked);
elasticitiesSubcritical = simplify(elasticityVars./A_subcritical.*areaGradientSubcritical);
table(elasticityLabels(:), elasticitiesChoked(:), ...
    'VariableNames', {'Input', 'Elasticity'})
```

ans = 7×2 table

	Input	Elasticity
1	"Mass flow"	1
2	"Upstream absolute pressure"	-1
3	"Backpressure"	0
4	"Absolute temperature"	1/2
5	"Molecular weight"	-1/2
6	"Heat-capacity ratio"	(k - 2*k*log(k + 1) + 2*k*log(2) - 1)/(2*(k - 1)^2)
7	"Discharge coefficient"	-1

In choked flow, backpressure has zero local sensitivity. That should not be generalized to all scenarios, because the subcritical branch depends on backpressure.

8. Generate reusable MATLAB functions

Use `matlabFunction` to create small numeric kernels. The generated functions are unit-free by design; their inputs are SI magnitudes produced by the unit-audit section.

```
matlabFunction(criticalPressureRatioExpr, ...
    'File', 'criticalPressureRatio', ...
    'Vars', k, ...
    'Optimize', true);
matlabFunction(P2/P1 <= criticalPressureRatioExpr, ...
    'File', 'reliefIsChoked', ...
    'Vars', [P1 P2 k], ...
    'Optimize', true);
matlabFunction(A_required, ...
    'File', 'reliefAreaGas', ...
    'Vars', [P1 P2 T1 MW R_u k Cd mdot_req], ...
    'Optimize', true);
matlabFunction(A_choked, ...
    'File', 'reliefAreaGasChoked', ...
    'Vars', [P1 T1 MW R_u k Cd mdot_req], ...
    'Optimize', true);
```

```
matlabFunction(elasticitiesChoked, ...
    'File', 'reliefSizingElasticityChoked', ...
    'Vars', [P1 P2 T1 MW R_u k Cd mdot_req], ...
    'Optimize', true);
matlabFunction(elasticitiesSubcritical, ...
    'File', 'reliefSizingElasticitySubcritical', ...
    'Vars', [P1 P2 T1 MW R_u k Cd mdot_req], ...
    'Optimize', true);
generatedFiles = ["criticalPressureRatio.m"; "reliefIsChoked.m"; "reliefAreaGas.m"; ...
    "reliefAreaGasChoked.m"; "reliefSizingElasticityChoked.m"; ...
    "reliefSizingElasticitySubcritical.m"]
```

```
generatedFiles = 6×1 string
"criticalPressureRatio.m"
"reliefIsChoked.m"
"reliefAreaGas.m"
"reliefAreaGasChoked.m"
"reliefSizingElasticityChoked.m"
"reliefSizingElasticitySubcritical.m"
```

9. Convert upset scenarios to numeric inputs

The downstream workflow starts from plant-style units and converts each row to the SI magnitudes expected by the generated functions. The scenario names are representative upset labels for teaching.

```
scenarioData = table(["Cooling water failure"; "External fire"; "Blocked outlet"; "High backpressure"], ...
    [150; 180; 165; 150], ...
    [15; 30; 25; 110], ...
    [120; 230; 150; 130], ...
    [10000; 18000; 12500; 9000], ...
    [1.09; 1.07; 1.08; 1.09], ...
    'VariableNames', {'Scenario', 'P1_psig', 'P2_psig', 'T1_F', 'MassFlow_lbm_hr', 'k'});
nScenarios = height(scenarioData);
P1Val = zeros(nScenarios, 1);
P2Val = zeros(nScenarios, 1);
T1Val = zeros(nScenarios, 1);
mdotVal = zeros(nScenarios, 1);
toSI = @(quantity, targetUnit) double(separateUnits(unitConvert(quantity, targetUnit)));
```

```
toSIAbsTemp = @(quantity, targetUnit) double(separateUnits(unitConvert(quantity, targetUnit, 'Temperature',
'absolute')));
for idx = 1:nScenarios
    P1Val(idx) = toSI(scenarioData.P1_psig(idx)*u.psi + 1*u.atm, u.Pa);
    P2Val(idx) = toSI(scenarioData.P2_psig(idx)*u.psi + 1*u.atm, u.Pa);
    T1Val(idx) = toSIAbsTemp(scenarioData.T1_F(idx)*u.Fahrenheit, u.K);
    mdotVal(idx) = toSI(scenarioData.MassFlow_lbm_hr(idx)*u.lbm/u.hr, u.kg/u.s);
end
MWVal = toSI(molecularWeightWithUnits, u.kg/u.mol);
areaPerSquareInch = toSI(1*u.in^2, u.m^2);
kVal = scenarioData.k;
scenarioInputsSI = table(scenarioData.Scenario, P1Val, P2Val, T1Val, mdotVal, kVal, ...
    'VariableNames', {'Scenario', 'P1_Pa', 'P2_Pa', 'T1_K', 'mdot_kg_s', 'k'})
```

scenarioInputsSI = 4×6 table

	Scenario	P1_Pa	P2_Pa	T1_K	mdot_kg_s	k
1	"Cooling water failure"	1.1355e+06	2.0475e+05	322.0389	1.2600	1.0900
2	"External fire"	1.3424e+06	3.0817e+05	383.1500	2.2680	1.0700
3	"Blocked outlet"	1.2390e+06	2.7369e+05	338.7056	1.5750	1.0800
4	"High backpressure"	1.1355e+06	8.5975e+05	327.5944	1.1340	1.0900

10. Screen scenarios with generated sizing kernels

The symbolic expressions are no longer manipulated directly in this section. Each scenario row calls the generated numeric functions for regime classification and required area, which is the interface a spreadsheet replacement, design-review script, or dashboard would reuse.

```
pressureRatioVal = P2Val./P1Val;
criticalRatioVal = arrayfun(@(kInput) criticalPressureRatio(kInput), kVal);
isChokedVal = arrayfun(@(row) reliefIsChoked(P1Val(row), P2Val(row), kVal(row)), (1:nScenarios).');
requiredArea_m2 = arrayfun(@(row) reliefAreaGas(P1Val(row), P2Val(row), T1Val(row), MWVal, ...
    8.31446261815324, kVal(row), dischargeCoefficientVal, mdotVal(row)), (1:nScenarios).');
requiredArea_in2 = requiredArea_m2/areaPerSquareInch;
regime = strings(nScenarios, 1);
regime(isChokedVal) = "Choked";
regime(~isChokedVal) = "Subcritical";
```

```
screeningResults = table(scenarioData.Scenario, regime, pressureRatioVal, criticalRatioVal, requiredArea_in2, ...
    'VariableNames', {'Scenario', 'Regime', 'P2_over_P1', 'CriticalRatio', 'RequiredArea_in2'})
```

`screeningResults` = 4×5 table

	Scenario	Regime	P2_over_P1	CriticalRatio	RequiredArea_in2
1	"Cooling water failure"	"Choked"	0.1803	0.5868	0.6046
2	"External fire"	"Choked"	0.2296	0.5911	1.0110
3	"Blocked outlet"	"Choked"	0.2209	0.5889	0.7127
4	"High backpressure"	"Subcritical"	0.7571	0.5868	0.5981

11. Select a representative standard orifice

The continuous required area is not a purchaseable valve size. Select the smallest representative standard area that exceeds the required area plus sizing margin.

```
[selectedOrifices, standardOrifices] = selectStandardOrifice(requiredArea_in2, sizingMarginVal);
standardOrifices
```

`standardOrifices` = 14×2 table

	Orifice	Area_in2
1	"D"	0.1100
2	"E"	0.1960
3	"F"	0.3070
4	"G"	0.5030
5	"H"	0.7850
6	"J"	1.2870
7	"K"	1.8380
8	"L"	2.8530
9	"M"	3.6000
10	"N"	4.3400

	Orifice	Area_in2
11	"P"	6.3800
12	"Q"	11.0500
13	"R"	16
14	"T"	26

```
scenarioResults = table(scenarioData.Scenario, regime, requiredArea_in2, selectedOrifices.TargetArea_in2, ...
    selectedOrifices.Orifice, selectedOrifices.StandardArea_in2, ...
    100*(selectedOrifices.StandardArea_in2./requiredArea_in2 - 1), ...
    'VariableNames', {'Scenario', 'Regime', 'RequiredArea_in2', 'TargetArea_in2', ...
    'SelectedOrifice', 'SelectedArea_in2', 'InstalledMargin_pct'})
```

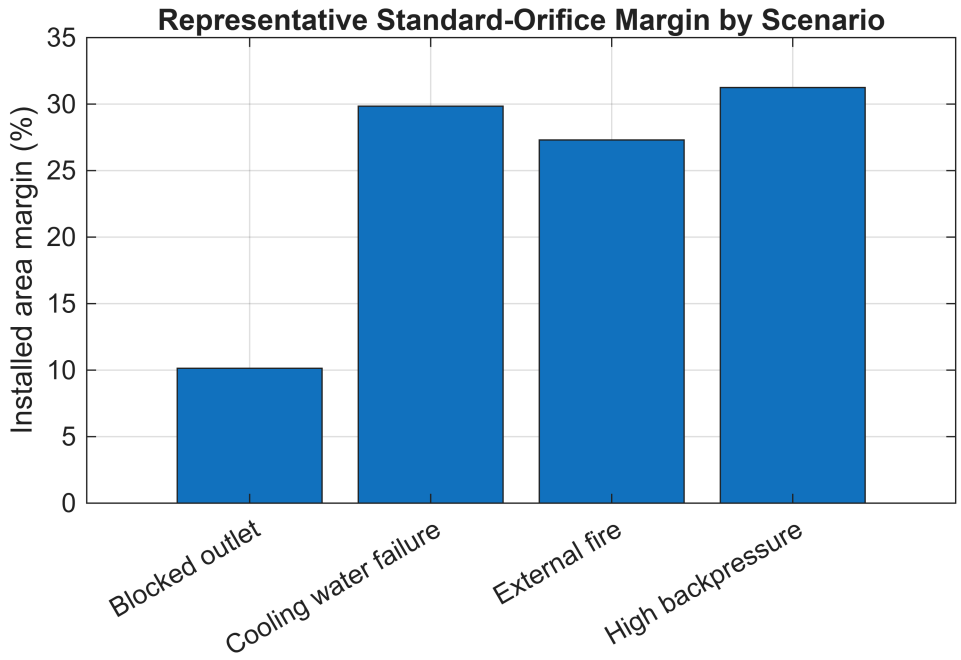
```
scenarioResults = 4×7 table
```

...

	Scenario	Regime	RequiredArea_in2	TargetArea_in2	SelectedOrifice	SelectedArea_in2
1	"Cooling water failure"	"Choked"	0.6046	0.6650	"H"	0.7850
2	"External fire"	"Choked"	1.0110	1.1121	"J"	1.2870
3	"Blocked outlet"	"Choked"	0.7127	0.7840	"H"	0.7850
4	"High backpressure"	"Subcritical"	0.5981	0.6579	"H"	0.7850

The standard-area table is representative for teaching. It should be checked against the governing standard before publication or engineering use. The chart below is just a quick visual review of the margin created by the discrete size selection.

```
bar(categorical(scenarioResults.Scenario), scenarioResults.InstalledMargin_pct)
ylabel("Installed area margin (%)")
title("Representative Standard-Orifice Margin by Scenario")
grid on
```



12. Evaluate sensitivities at the screened cases

The symbolic elasticities are evaluated after regime selection. The high-backpressure case uses the subcritical sensitivity expression, which restores backpressure as an active design variable.

```
elasticityValues = zeros(nScenarios, numel(elasticityLabels));
for idx = 1:nScenarios
    if isChokedVal(idx)
        elasticityValues(idx, :) = reliefSizingElasticityChoked(P1Val(idx), P2Val(idx), T1Val(idx), MWVal, ...
            8.31446261815324, kVal(idx), dischargeCoefficientVal, mdotVal(idx));
    else
        elasticityValues(idx, :) = reliefSizingElasticitySubcritical(P1Val(idx), P2Val(idx), T1Val(idx), MWVal,
...
            8.31446261815324, kVal(idx), dischargeCoefficientVal, mdotVal(idx));
    end
end
sensitivityResults = array2table(elasticityValues, ...
```

```
    'VariableNames', {'MassFlow', 'P1', 'P2', 'T1', 'MW', 'k', 'Cd'});
sensitivityResults.Scenario = scenarioData.Scenario;
sensitivityResults = movevars(sensitivityResults, 'Scenario', 'Before', 'MassFlow')
```

sensitivityResults = 4×8 table

	Scenario	MassFlow	P1	P2	T1	MW	k	Cd
1	"Cooling water failure"	1	-1	0	0.5000	-0.5000	-0.3677	-1
2	"External fire"	1	-1	0	0.5000	-0.5000	-0.3693	-1
3	"Blocked outlet"	1	-1	0	0.5000	-0.5000	-0.3685	-1
4	"High backpressure"	1	-1.8591	0.8591	0.5000	-0.5000	-0.1917	-1

13. Audit a choked-flow shortcut

This comparison asks a design-review question: would a simpler choked-only sizing equation give the same answer for every screened scenario? Compute each case with the choked expression and compare it with the regime-aware area from Section 10.

```
chokedOnlyArea_m2 = arrayfun(@(row) reliefAreaGasChoked(P1Val(row), T1Val(row), MWVal, ...
    8.31446261815324, kVal(row), dischargeCoefficientVal, mdotVal(row)), (1:nScenarios).');
chokedOnlyArea_in2 = chokedOnlyArea_m2/areaPerSquareInch;
chokedOnlyShortfall_pct = 100*(requiredArea_in2 - chokedOnlyArea_in2)./requiredArea_in2;
chokedOnlyShortfall_pct(abs(chokedOnlyShortfall_pct) < 1e-10) = 0;
assumptionCheck = table(scenarioData.Scenario, regime, requiredArea_in2, chokedOnlyArea_in2,
chokedOnlyShortfall_pct, ...
    'VariableNames', {'Scenario', 'CorrectRegime', 'CorrectArea_in2', 'ChokedOnlyArea_in2',
'ChokedOnlyShortfall_pct'})
```

assumptionCheck = 4×5 table

	Scenario	CorrectRegime	CorrectArea_in2	ChokedOnlyArea_in2	ChokedOnlyShortfall_pct
1	"Cooling water failure"	"Choked"	0.6046	0.6046	0
2	"External fire"	"Choked"	1.0110	1.0110	0
3	"Blocked outlet"	"Choked"	0.7127	0.7127	0
4	"High backpressure"	"Subcritical"	0.5981	0.5488	8.2479

For the high-backpressure case, the choked-only shortcut under-sizes the required area. The value of this check is traceability: the symbolic derivation produced both branches, and the downstream screen shows why the pressure-ratio logic should remain in the reusable sizing function.

14. Run a numerical margin sweep

A downstream sizing review would not manipulate symbolic expressions. It would call `reliefAreaGas` repeatedly over numeric cases. Here the governing scenario's selected orifice is checked over a grid of required mass flow and relieving temperature assumptions.

```
[~, governingScenarioIdx] = max(requiredArea_in2);
flowScale = linspace(0.95, 1.15, 41);
temperatureOffset_K = linspace(-10, 20, 41);
[flowScaleGrid, temperatureOffsetGrid] = ndgrid(flowScale, temperatureOffset_K);
areaSweep_m2 = arrayfun(@(flowFactor, temperatureOffset) reliefAreaGas( ...
    P1Val(governingScenarioIdx), P2Val(governingScenarioIdx), ...
    T1Val(governingScenarioIdx) + temperatureOffset, MWVal, ...
    8.31446261815324, kVal(governingScenarioIdx), dischargeCoefficientVal, ...
    mdotVal(governingScenarioIdx)*flowFactor), flowScaleGrid, temperatureOffsetGrid);
areaSweep_in2 = areaSweep_m2/areaPerSquareInch;
installedArea_in2 = scenarioResults.SelectedArea_in2(governingScenarioIdx);
marginSweep_pct = 100*(installedArea_in2./areaSweep_in2 - 1);
[minimumMargin_pct, worstCaseLinearIdx] = min(marginSweep_pct(:));
[worstFlowIdx, worstTemperatureIdx] = ind2sub(size(marginSweep_pct), worstCaseLinearIdx);
marginSweepSummary = table(scenarioData.Scenario(governingScenarioIdx), ...
    scenarioResults.SelectedOrifice(governingScenarioIdx), installedArea_in2, ...
    max(areaSweep_in2, [], "all"), minimumMargin_pct, ...
    100*(flowScale(worstFlowIdx) - 1), temperatureOffset_K(worstTemperatureIdx), ...
    'VariableNames', {'GoverningScenario', 'InstalledOrifice', 'InstalledArea_in2', ...
    'MaxRequiredArea_in2', 'MinimumMargin_pct', 'WorstFlowChange_pct', 'WorstTemperatureOffset_K'})
```

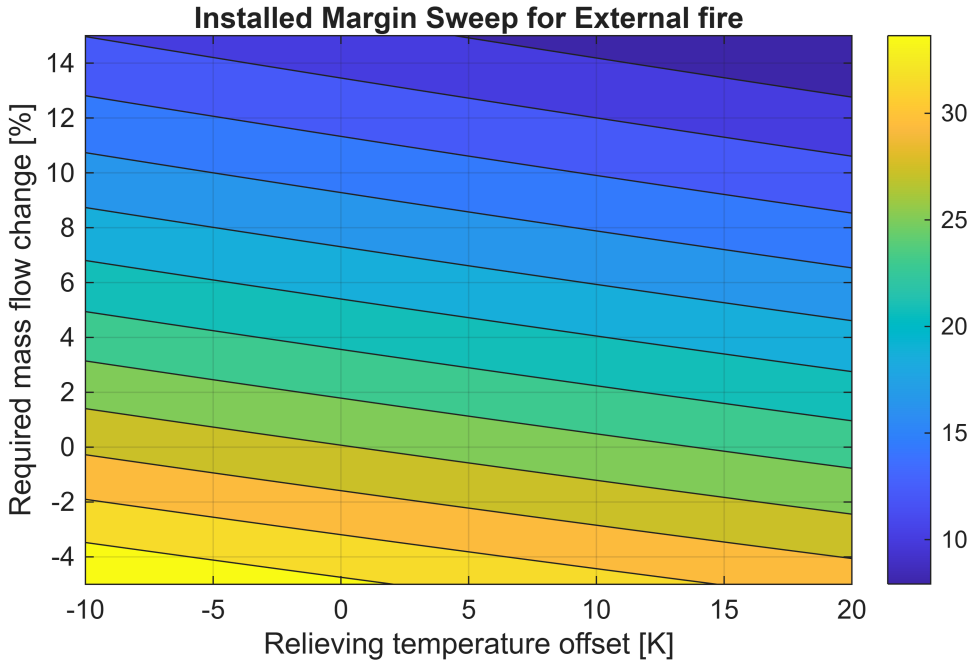
```
marginSweepSummary = 1×7 table
```

...

	GoverningScenario	InstalledOrifice	InstalledArea_in2	MaxRequiredArea_in2	MinimumMargin_pct
1	"External fire"	"J"	1.2870	1.1926	7.9179

```
contourf(temperatureOffsetGrid, 100*(flowScaleGrid - 1), marginSweep_pct, 12)
colorbar
```

```
xlabel("Relieving temperature offset [K]")
ylabel("Required mass flow change [%]")
title("Installed Margin Sweep for " + scenarioData.Scenario(governingScenarioIdx))
grid on
```



The plot is the downstream value of the generated function: a compact numerical margin map from the symbolic sizing equation. Negative margin would identify combinations where the selected standard orifice no longer covers the required area.

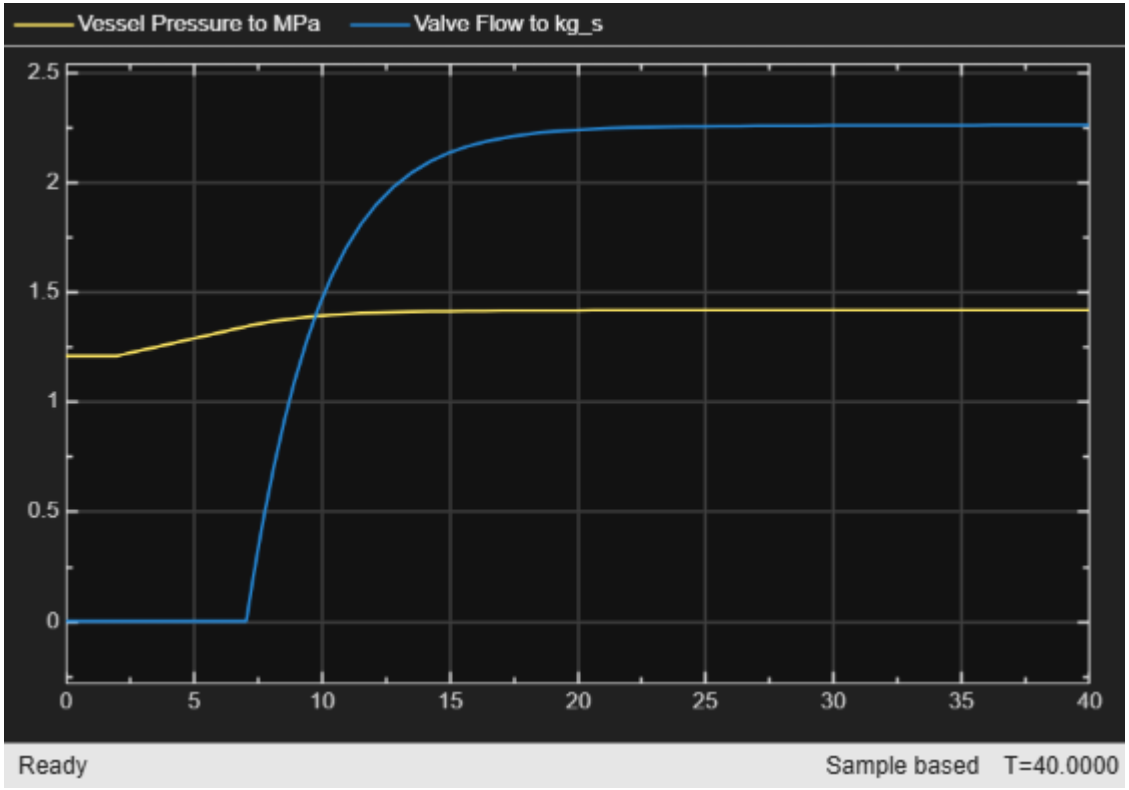
15. Configure the Simscape Fluids model

The prepared Simscape Fluids model is a small dynamic vessel-relief sketch. A controlled inlet mass-flow upset pressurizes a constant-volume vessel, and the relief valve vents to a downstream reservoir. This section updates only the `Pressure Relief Valve (G)` block from the symbolic sizing workflow.

```
simscapeModel = "ReliefValveSizingSimscape";
reliefValveBlock = simscapeModel + "/Pressure Relief Valve (G)";
[~, simscapeScenarioIdx] = max(requiredArea_in2);
```

```
selectedOrificeArea_m2 = scenarioResults.SelectedArea_in2(simscapeScenarioIdx)*areaPerSquareInch;
setPressureDifference_MPa = (P1Val(simscapeScenarioIdx) - P2Val(simscapeScenarioIdx))/1e6;
simscapeConfiguredParameters = table(strings(0, 1), strings(0, 1), strings(0, 1), ...
    'VariableNames', {'Block', 'Parameter', 'StoredExpression'});
simscapeResponseSummary = table(strings(0, 1), zeros(0, 1), strings(0, 1), ...
    'VariableNames', {'Metric', 'Value', 'Unit'});
simscapeModelStatus = "Skipped: " + simscapeModel + ".slx is not on the MATLAB path.";
if exist(simscapeModel + ".slx", "file") == 4
    try
        open_system(simscapeModel)
        set_param(reliefValveBlock, ...
            "opening_pressure_spec", "fluids.gas.valves_orifices.enum.PressureControlSpec.PressureDifference",
...
            "valve_parameterization", "fluids.gas.valves_orifices.enum.OrificeParameterization.Area", ...
            "orifice_area_max", compose("%.16g", selectedOrificeArea_m2), ...
            "orifice_area_max_unit", "m^2", ...
            "Cd", compose("%.16g", dischargeCoefficientVal), ...
            "p_set_differential", compose("%.16g", setPressureDifference_MPa), ...
            "p_set_differential_unit", "MPa", ...
            "p_range", "0.1", ...
            "p_range_unit", "MPa")
        simscapeConfiguredParameters = table([reliefValveBlock; reliefValveBlock; reliefValveBlock;
reliefValveBlock], ...
            ["orifice_area_max"; "Cd"; "p_set_differential"; "p_range"], ...
            [string(get_param(reliefValveBlock, "orifice_area_max")); ...
            string(get_param(reliefValveBlock, "Cd")); ...
            string(get_param(reliefValveBlock, "p_set_differential")); ...
            string(get_param(reliefValveBlock, "p_range"))], ...
            'VariableNames', {'Block', 'Parameter', 'StoredExpression'})
        in = Simulink.SimulationInput(simscapeModel);
        in = in.setModelParameter("StopTime", "40");
        simscapeOut = sim(in);
        vesselPressureTrace = simscapeOut.vesselPressure_MPa;
        reliefMassFlowTrace = simscapeOut.reliefMassFlow_kg_s;
        vesselPressure_MPa = squeeze(vesselPressureTrace.Data);
        reliefMassFlow_kg_s = squeeze(reliefMassFlowTrace.Data);
        simscapeResponseSummary = table(["Initial vessel pressure"; "Peak vessel pressure"; ...
```

```
            "Final vessel pressure"; "Final relief mass flow"], ...
            [vesselPressure_MPa(1); max(vesselPressure_MPa); vesselPressure_MPa(end); reliefMassFlow_kg_s(end)],
...
            ["MPa"; "MPa"; "MPa"; "kg/s"], ...
            'VariableNames', {'Metric', 'Value', 'Unit'})
        tiledlayout(2, 1)
        nexttile
        plot(vesselPressureTrace.Time, vesselPressure_MPa, "LineWidth", 1.2)
        ylabel("Vessel pressure [MPa]")
        grid on
        nexttile
        plot(reliefMassFlowTrace.Time, reliefMassFlow_kg_s, "LineWidth", 1.2)
        xlabel("Time [s]")
        ylabel("Relief mass flow [kg/s]")
        grid on
        simscapeModelStatus = "Updated " + reliefValveBlock + " and simulated the vessel relief response."
    catch exception
        simscapeModelStatus = "Skipped: " + string(exception.message)
    end
end
```



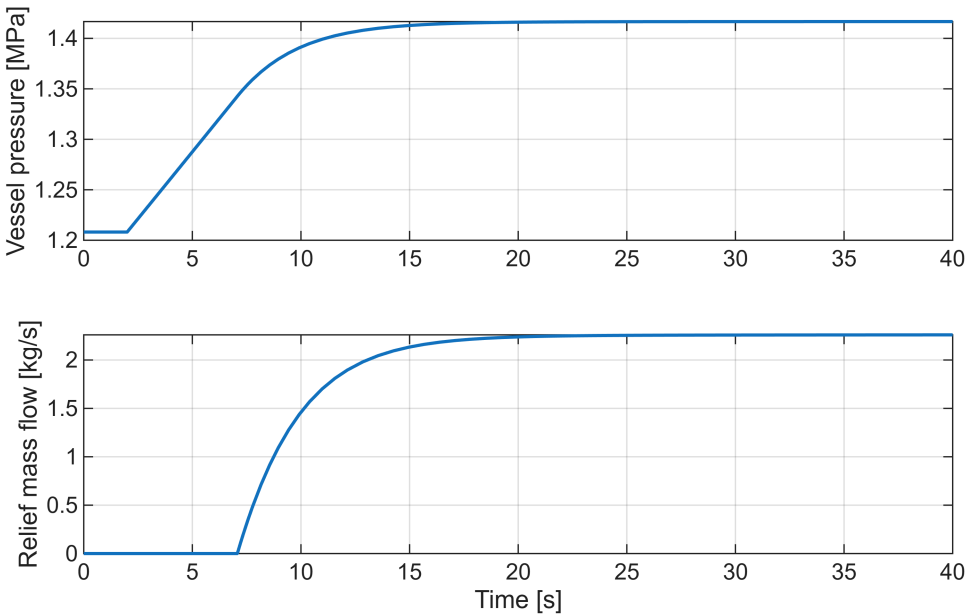
`simscapeConfiguredParameters = 4×3 table`

	Block	Parameter	StoredExpression
1	"ReliefValveSizingSimscape/Pressure Relief Valve (G)"	"orifice_area_max"	"0.00083032092"
2	"ReliefValveSizingSimscape/Pressure Relief Valve (G)"	"Cd"	"0.975"
3	"ReliefValveSizingSimscape/Pressure Relief Valve (G)"	"p_set_differential"	"1.034213593975254"
4	"ReliefValveSizingSimscape/Pressure Relief Valve (G)"	"p_range"	"0.1"

`simscapeResponseSummary = 4×3 table`

	Metric	Value	Unit
1	"Initial vessel pressure"	1.2081	"MPa"
2	"Peak vessel pressure"	1.4166	"MPa"
3	"Final vessel pressure"	1.4166	"MPa"

	Metric	Value	Unit
4	"Final relief mass flow"	2.2599	"kg/s"




```
simscapeModelStatus =
"Updated ReliefValveSizingSimscape/Pressure Relief Valve (G) and simulated the vessel relief response."
```